

Aggregazione

Funzioni aggregate

COUNT, SUM, AVG, MIN e MAX per riassumere molte righe in un solo risultato.

Quando non vuoi vedere ogni riga

A volte non ti interessa leggere tutti i dati uno per uno. Vuoi un riassunto.

Per esempio:

- quanti ordini ci sono?
- quanto abbiamo incassato?
- qual è il prezzo più alto?
- qual è la media dei voti?

Le **funzioni aggregate** servono proprio a questo: prendono molte righe e restituiscono un risultato riassuntivo.

Contare le righe con COUNT

Questa query conta quante righe ci sono nella tabella `ordini` :

```
SELECT COUNT(*)  
FROM ordini;
```

Se la tabella contiene 125 ordini, il risultato sarà un numero solo: `125` .

`COUNT(*)` significa "conta le righe".

Le funzioni più usate

Ecco le aggregate che incontrerai più spesso:

Funzione	Cosa fa
COUNT ()	conta righe o valori
SUM ()	somma numeri
AVG ()	calcola una media
MIN ()	trova il valore più basso
MAX ()	trova il valore più alto

Alcuni esempi

Per sommare il totale degli ordini:

```
SELECT SUM(totale)
FROM ordini;
```

Per calcolare il prezzo medio dei prodotti:

```
SELECT AVG(prezzo)
FROM prodotti;
```

Per trovare il prezzo più alto:

```
SELECT MAX(prezzo)
FROM prodotti;
```

Per trovare il prezzo più basso:

```
SELECT MIN(prezzo)
FROM prodotti;
```

Ogni query restituisce un solo valore, perché stai chiedendo un riassunto.

Dare un nome al risultato

Il nome automatico della colonna può essere poco leggibile. Per questo puoi usare **AS** :

```
SELECT COUNT(*) AS numero_ordini
FROM ordini;
```

Ora il risultato avrà una colonna chiamata **numero_ordini** .

È un dettaglio piccolo, ma rende le query molto più chiare, soprattutto nei report.

Attenzione a **NULL**

NULL significa "dato mancante".

Le aggregate lo trattano in modo particolare:

- **COUNT(*)** conta le righe, anche se alcune colonne hanno valori mancanti.
- **COUNT(colonna)** conta solo le righe in cui quella colonna ha un valore.
- **SUM()** , **AVG()** , **MIN()** e **MAX()** di solito ignorano i valori **NULL** .

La cosa importante da ricordare è questa: **mancante non significa zero**.

Riepilogo rapido

- Le funzioni aggregate riassumono molte righe.
- **COUNT(*)** conta le righe.
- **SUM()** somma, **AVG()** calcola la media.
- **MIN()** e **MAX()** trovano il valore più basso e più alto.
- Con **AS** puoi dare un nome leggibile al risultato.

CAST e CONVERT

Cambiare il tipo di un valore quando serve adattarlo al contesto.

A volte il valore è giusto, ma la forma no

Capita di avere un valore corretto nel contenuto, ma nel tipo sbagliato per l'operazione che vuoi fare. In questi casi puoi convertirlo.

`CAST` e, in alcuni database, `CONVERT`, servono proprio a cambiare il tipo di un valore.

```
SELECT CAST(totale AS INT)
FROM ordini;
```

Questa query prende il valore `totale` e lo converte in intero.

Perché può servire

Le conversioni sono utili quando:

- devi confrontare valori che arrivano con tipi diversi
- vuoi formattare un risultato
- vuoi troncare o adattare un valore
- stai lavorando con dati importati in modo poco uniforme

`CAST` e `CONVERT` non sono gemelli perfetti

`CAST` è più standard e più portabile. Se vuoi scrivere SQL che assomigli il più possibile a quello universale, di solito è la scelta migliore.

`CONVERT` esiste in diversi dialetti, ma non sempre con la stessa sintassi. Per questo conviene usarlo sapendo su quale database ti trovi.

Funzioni su date e tempo

Lavorare con date, orari e timestamp in modo più preciso.

Il tempo nei database conta tantissimo

Date e orari compaiono ovunque. Ordini, iscrizioni, appuntamenti, scadenze, ultime modifiche: quasi ogni applicazione ha bisogno di ricordare quando è successo qualcosa.

Per questo SQL offre funzioni dedicate al tempo.

```
SELECT CURRENT_DATE;
```

```
SELECT CURRENT_TIMESTAMP;
```

La prima query restituisce la data di oggi. La seconda restituisce data e ora correnti.

Un uso tipico: estrarre una parte della data

Molti database permettono di estrarre anno, mese o giorno da una data:

```
SELECT EXTRACT(YEAR FROM data_ordine)
FROM ordini;
```

Qui stai chiedendo solo l'anno della colonna `data_ordine`.

Quando ti tornano utili

Le funzioni su date e tempo servono spesso per:

- filtrare ordini recenti
- raggruppare per mese o anno
- calcolare scadenze
- capire quando un dato è stato creato o modificato

Appena lavori con dati reali, queste funzioni smettono subito di essere opzionali.

GROUP BY

Come raggruppare le righe che condividono lo stesso valore.

Ragionare per gruppi

GROUP BY serve quando non vuoi guardare le righe una per una, ma vuoi **raggrupparle**.

Immagina una pila di schede clienti. Invece di leggerle in ordine, le dividi per città: Roma da una parte, Milano da un'altra, Torino da un'altra ancora.

SQL fa la stessa cosa con **GROUP BY**.

```
SELECT città, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY città;
```

Questa query dice:

"Raggruppa i clienti per città e conta quanti clienti ci sono in ogni città."

Un esempio con una tabella

Partiamo da questi dati:

nome	città
Luca	Roma
Marta	Milano
Sara	Roma

La query:

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta;
```

produce:

citta	numero_clienti
Milano	1
Roma	2

SQL crea un gruppo per ogni città. Poi `COUNT(*)` conta quante righe ci sono in ciascun gruppo.

Perché va spesso insieme alle aggregate

`GROUP BY` da solo crea i gruppi. Le funzioni aggregate fanno i calcoli dentro quei gruppi.

Per esempio:

- `COUNT(*)` conta quante righe ci sono nel gruppo.
- `SUM(totale)` somma i totali del gruppo.
- `AVG(prezzo)` calcola la media del gruppo.

Per questo `GROUP BY` è così usato nei report.

Una regola da ricordare

Quando usi `GROUP BY`, nel `SELECT` metti di solito:

- la colonna con cui stai raggruppando
- una o più funzioni aggregate

Questa query è chiara:

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta;
```

Questa invece crea confusione:

```
SELECT citta, nome, COUNT(*)  
FROM clienti  
GROUP BY citta;
```

Il problema è `nome`. Se a Roma ci sono Luca e Sara, quale nome dovrebbe mostrare SQL per il gruppo "Roma"?

Quando raggruppi, stai lasciando il dettaglio delle singole righe. Per questo devi mostrare il gruppo e i calcoli sul gruppo.

Raggruppare per più colonne

Puoi anche raggruppare usando più colonne:

```
SELECT citta, stato, COUNT(*) AS numero_clienti  
FROM clienti  
GROUP BY citta, stato;
```

Qui SQL crea un gruppo per ogni combinazione di `citta` e `stato`.

È come dividere prima le schede per città e poi, dentro ogni città, separarle per stato.

Riepilogo rapido

- `GROUP BY` raggruppa righe con lo stesso valore.
- Si usa spesso con `COUNT()`, `SUM()`, `AVG()`, `MIN()` e `MAX()`.
- Nel `SELECT` mostri le colonne di raggruppamento e i calcoli aggregati.
- Puoi raggruppare anche per più colonne.

HAVING

Come filtrare i gruppi creati da GROUP BY dopo averli costruiti.

Filtrare dopo il raggruppamento

HAVING assomiglia a **WHERE** , ma lavora in un momento diverso.

WHERE filtra le righe prima di creare i gruppi. **HAVING** filtra i gruppi dopo che **GROUP BY** li ha creati.

Immagina di dividere gli studenti per classe. Prima crei i gruppi. Poi tieni solo le classi con almeno 20 studenti. Quel secondo filtro è il lavoro di **HAVING** .

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta
HAVING COUNT(*) >= 2;
```

Questa query mostra solo le città con almeno due clienti.

Un esempio piccolo

Partiamo da questa tabella:

nome	citta
Luca	Roma
Sara	Roma
Marta	Milano

La query:

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta
HAVING COUNT(*) >= 2;
```

produce:

citta	numero_clienti
Roma	2

Milano non appare perché il suo gruppo contiene un solo cliente.

WHERE e HAVING insieme

Puoi usare entrambi nella stessa query:

```
SELECT citta, COUNT(*) AS numero_clienti_attivi
FROM clienti
WHERE attivo = TRUE
GROUP BY citta
HAVING COUNT(*) >= 2;
```

Qui succedono due cose:

1. `WHERE attivo = TRUE` tiene solo i clienti attivi.
2. `HAVING COUNT(*) >= 2` tiene solo le città con almeno due clienti attivi.

Prima filtri le righe. Poi crei i gruppi. Poi filtri i gruppi.

Quando ti serve HAVING

Usa `HAVING` quando la condizione riguarda un risultato aggregato:

- città con più di 10 clienti
- prodotti con vendite sopra una certa soglia
- mesi con almeno 100 ordini
- categorie con prezzo medio superiore a 50

Se nella condizione compare `COUNT()`, `SUM()`, `AVG()` o una funzione simile, probabilmente ti serve `HAVING`.

Riepilogo rapido

- `WHERE` filtra righe singole.
- `HAVING` filtra gruppi.
- `HAVING` si usa dopo `GROUP BY`.
- Si usa spesso con funzioni aggregate come `COUNT()`, `SUM()` e `AVG()`.

Funzioni su stringhe

Operazioni comuni sui testi, come maiuscole, minuscole e concatenazione.

Non sempre il testo va lasciato così com'è

Le stringhe sono i valori testuali. Nomi, email, città, codici, descrizioni: in un database il testo è ovunque.

SQL offre funzioni che ti aiutano a trasformarlo o combinarlo.

```
SELECT UPPER(nome) FROM clienti;
```

Questa query trasforma i nomi in maiuscolo.

Alcune funzioni che vedrai spesso

Funzioni comuni:

- UPPER()
- LOWER()
- LENGTH()
- TRIM()
- CONCAT()

Due esempi che parlano da soli

Per togliere spazi inutili:

```
SELECT TRIM(nome)  
FROM clienti;
```

Per unire due pezzi di testo:

```
SELECT CONCAT(nome, ' - ', citta)
FROM clienti;
```

Questa seconda query costruisce una piccola etichetta, per esempio **Luca – Roma** .

Quando servono davvero

Le funzioni sulle stringhe sono utili quando:

- vuoi pulire dati inseriti male
- vuoi confrontare testi in modo più uniforme
- vuoi costruire un'etichetta leggibile
- vuoi fare piccoli controlli sulla lunghezza del contenuto

Sono strumenti piccoli, ma diventano molto preziosi nella pratica quotidiana.